Chapter 2: Integrating quality activities in the project life cycle

Reference

This chapter refers from the book:

Mastering Software Quality Assurance: Best Practices, Tools and Techniques for Software Developers

Introduction to Software Testing

Outline

2.1. Software development methodologies

2.2. Testing level

- 2.2.1. Introduction
- 2.2.2. Unit test
- 2.2.3. Integration test
- 2.2.4. System test
- 2.2.5. Acceptance test

2.3. A model of SQA defect removal effectiveness and cost.

2.4. Create SQA plan

The SDLC

- The ‘classic mode.’
  - Still in use today.
  - Captures the major building blocks in development
  - Linear sequence
  - Highly structured; plan-driven; Heavy-weight process
  - Product delivered for evaluation and deployment at the end of development and testing
  - Big bang approach
  - Used for major projects of length
  - But serves as a framework for other models…

Prototyping Model

- Replaces some of the parts of the SDLC with an evolutionary and iterative process.
- Software prototypes are **repeatedly** **provided** to customer for evaluation and feedback.
  - Primarily iterate design and implementation.
  - Development team provided requirements.
- Ultimately, the product reaches a satisfactory completion.
- Then, the remainder of the process is carried out in the context of another model, such as SDLC

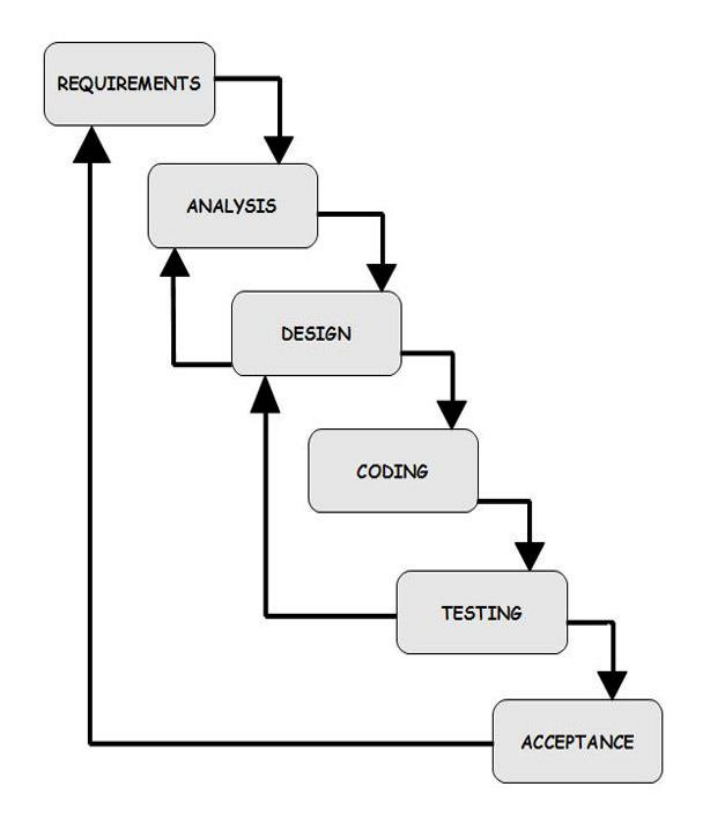
Spiral Model

- Uses an iterative approach designed to address each phases in development by obtaining **customer comments and change, risk analysis, and resolution.**
- The spiral model typically has a 'spiral' for each of the traditional development phases.
- Within a cycle, **specific engineering** (design, development, etc.) can take place using any other models, like SDLC, prototyping,..
- The Spiral Model (Barry Boehm) is a **risk-centered** development model where each spiral includes major risk activities / assessments.
- Was developed after SDLC in response to delayed risk in SDLC
- As the SDLC, it is considered a heavy-weight, plan-driven methodology and is highly structured.

The Object-Oriented Model

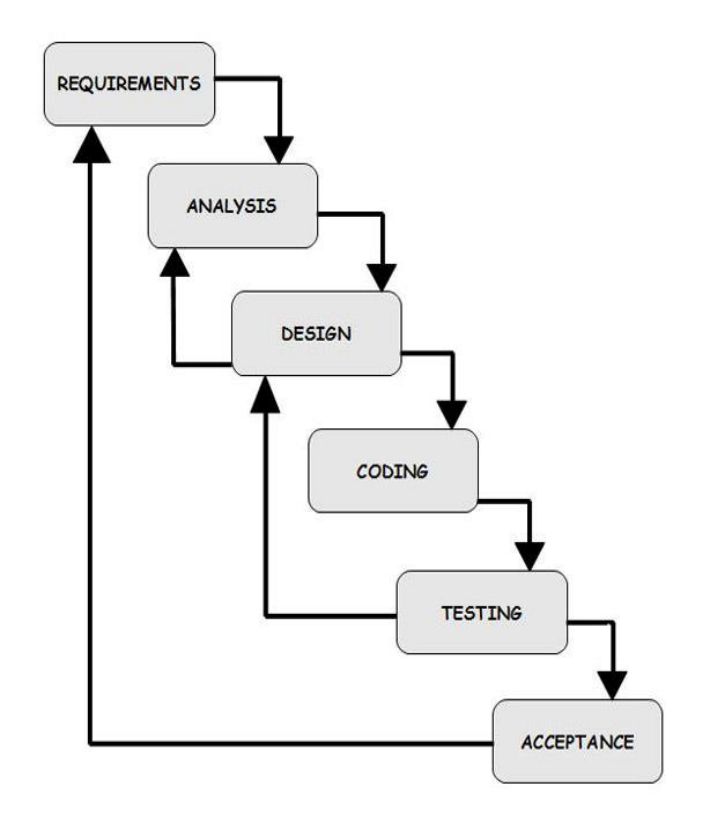
- Emphasis here is re-usability via reusable objects and components.
- Component-based software development.
- For non-available components, developer may
  - prototype needed modules,
  - use an SDLC approach,
  - purchase libraries of objects,
  - develop ‘his’ own, etc.

The SDLC



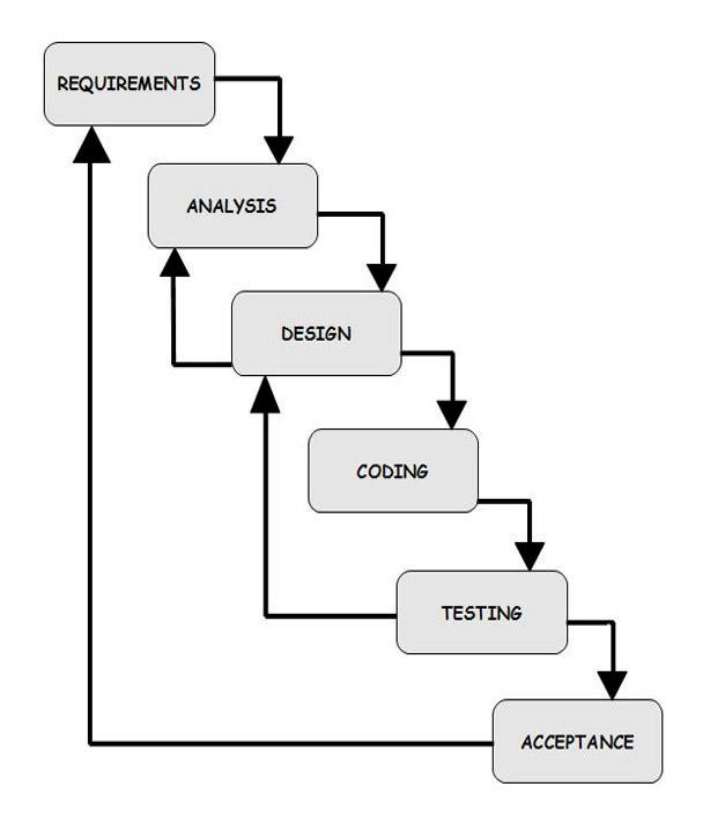
- **Requirements Definition**: done by customers
- **Analysis**: analyze requirements to form an initial software model
- **Design**: Detailed definition of inputs/outputs and processes including data structures, software structure, etc.

The SDLC



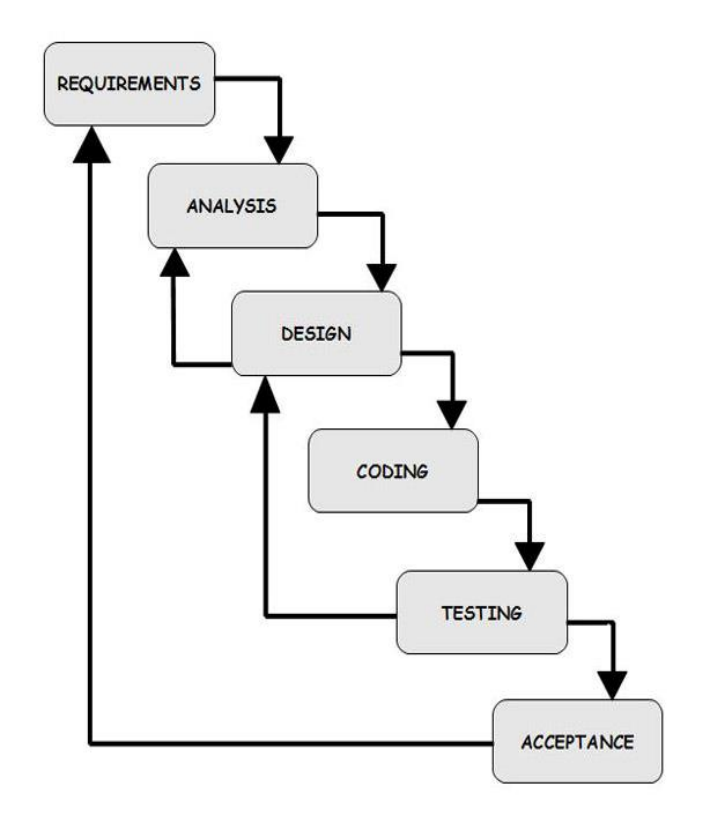
- **Coding**: Design translated into code.
  - Coding includes SQA activities such as inspections, unit tests and integration tests
  - Many takeoffs from this: These tests done by developers: individual (unit), group or team (integration tests….)

The SDLC



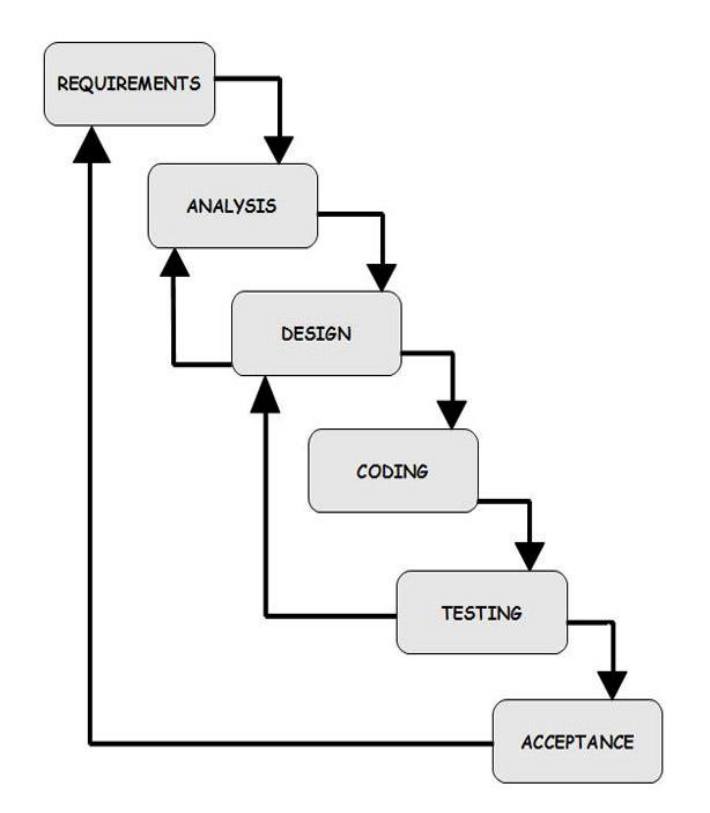
- **System Tests**: Goal: to discover errors / correct errors to achieve an acceptable level of quality. Carried out by developers prior to delivery.
- Sometimes 'acceptance tests' carried out by customer or in conjunction with developer

The SDLC



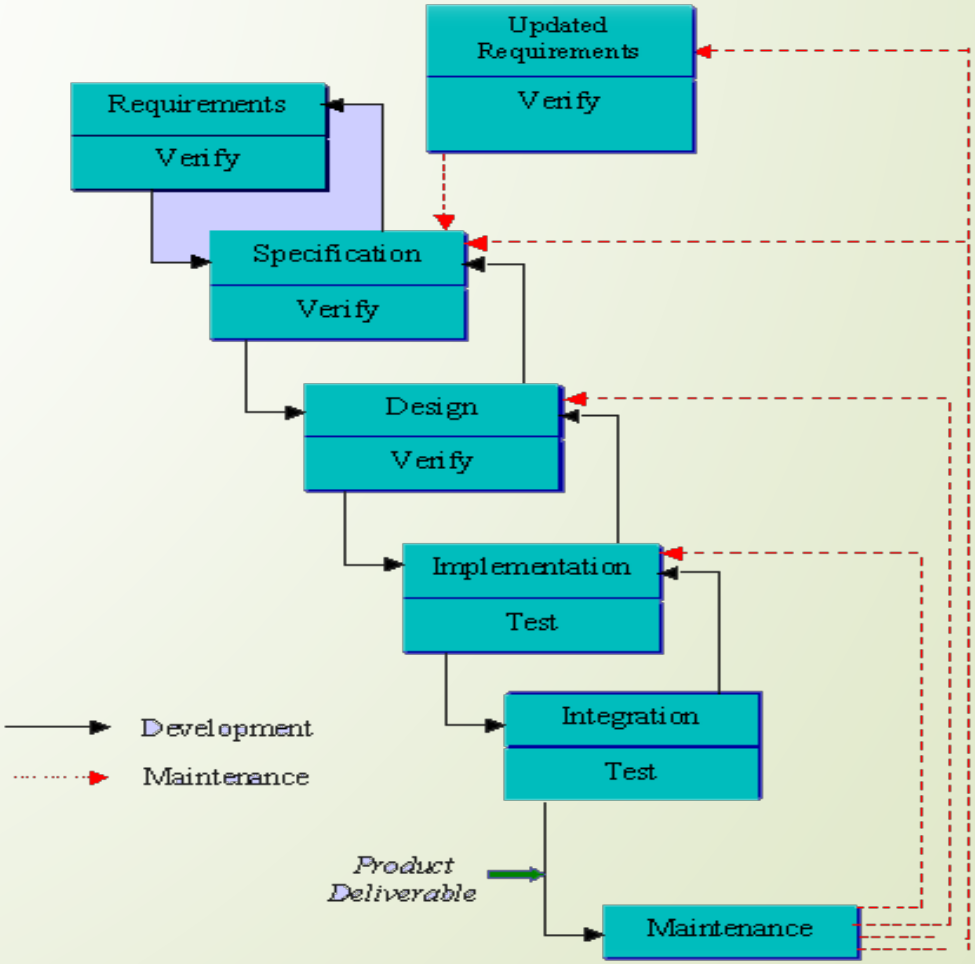
- **Installation / Conversion**:
  - After testing, system is installed and/or replaces an existing system;
  - Requires software / data conversion
  - Important to not interrupt daily activities during conversion process.
- Install incrementally, run in parallel; turn switch and live with it, etc.

The SDLC

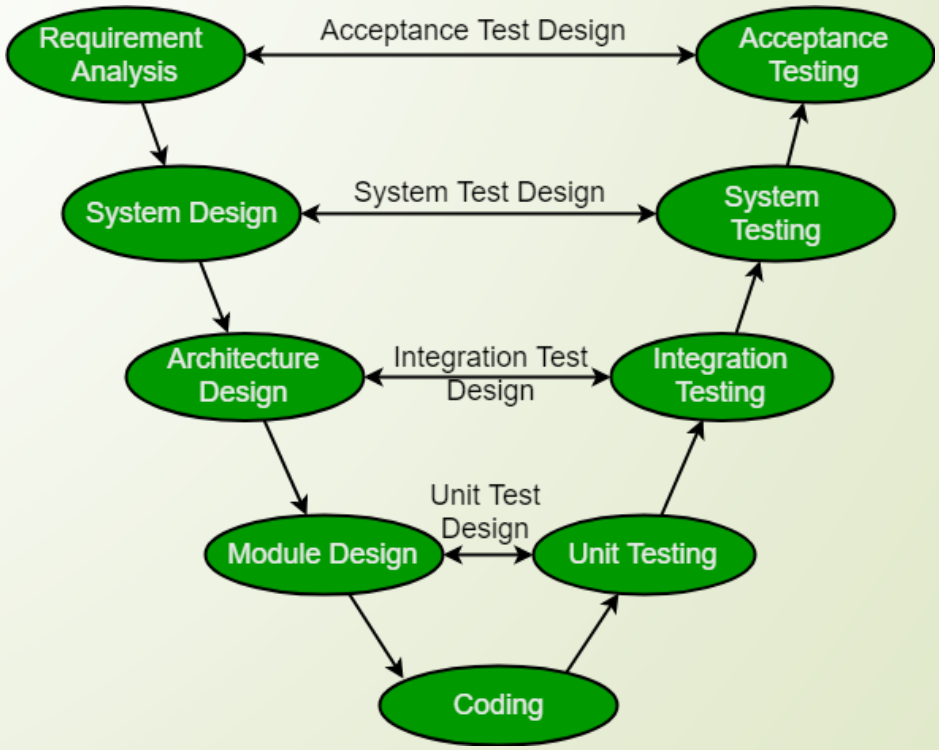


- **Operations and Maintenance**:
  - Hopefully done for years.
  - Maintenance:
    - Corrective
    - Adaptive
    - Perfective
- Lots of variations to the classic SDLC many in response to problems….
- Notice the feedback loops?

The SDLC


Updated Requirements
Verify
Requirements
Verify
Specification
Verify
Design
Verify
Implementation
Test
Integration
Test
Maintenance
Development
Maintenance
Product Deliverable

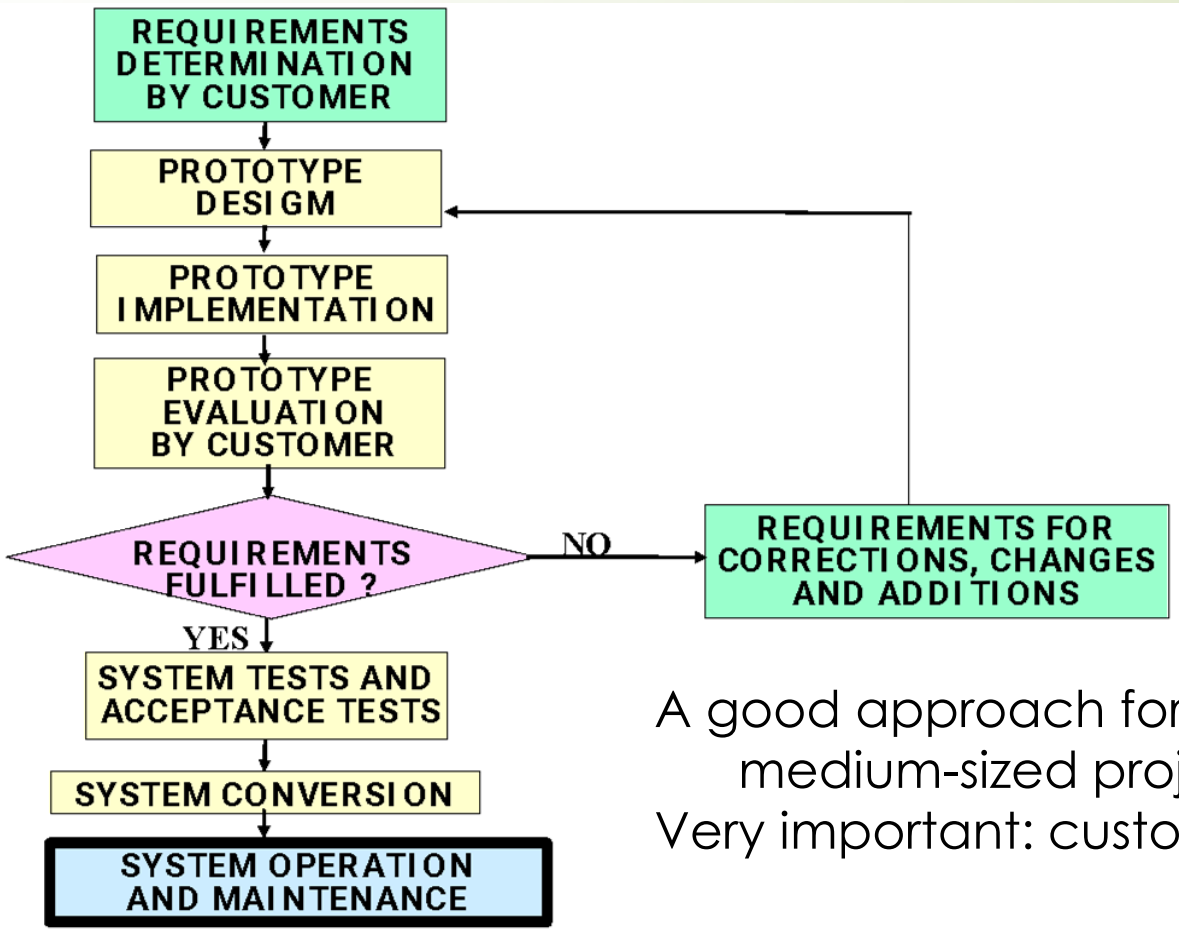
“V” Model


Requirement Analysis
Acceptance Test Design
Acceptance Testing
System Design
System Test Design
System Testing
Architecture Design
Integration Test Design
Integration Testing
Unit Test Design
Module Design
Unit Testing
Coding

The Prototyping Model

- One main idea behind prototyping is for the development of fast prototypes and customer availability for feedback.
- Often prototyping tools are used to help
- Developers respond to feedback and add additional parts as application evolves into an acceptable product.
- Recognize this process can be **inserted** into the SDLC or other models.

The Prototyping Model

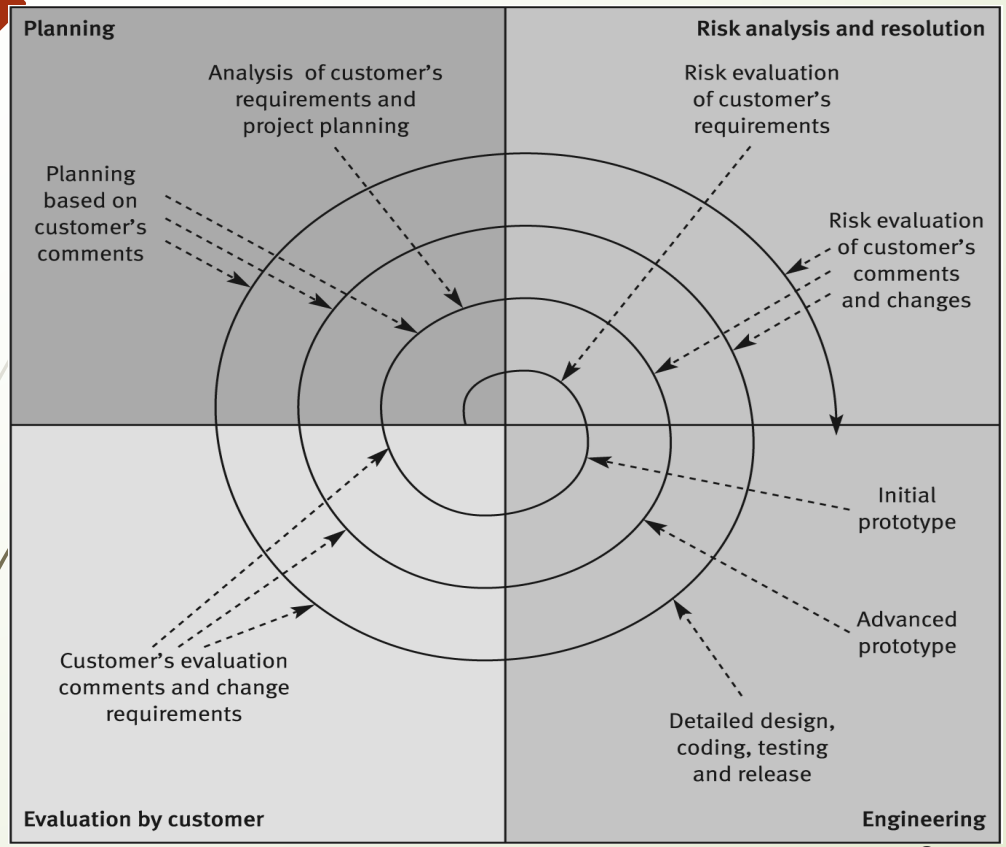


A good approach for small to medium-sized projects.
Very important: customer involvement.

The Spiral Model

- A heavy-weight, plan-driven, highly-structured approach for large projects.
- Especially designed for those with higher chances of failure.
- Combines iterative model, emphasizes risk assessment, customer participation, prototyping, and more
- Definitely an iterative process.

The Spiral Model



Source: After Boehm 1988 (© 1988 IEEE)

Can see each spiral includes:

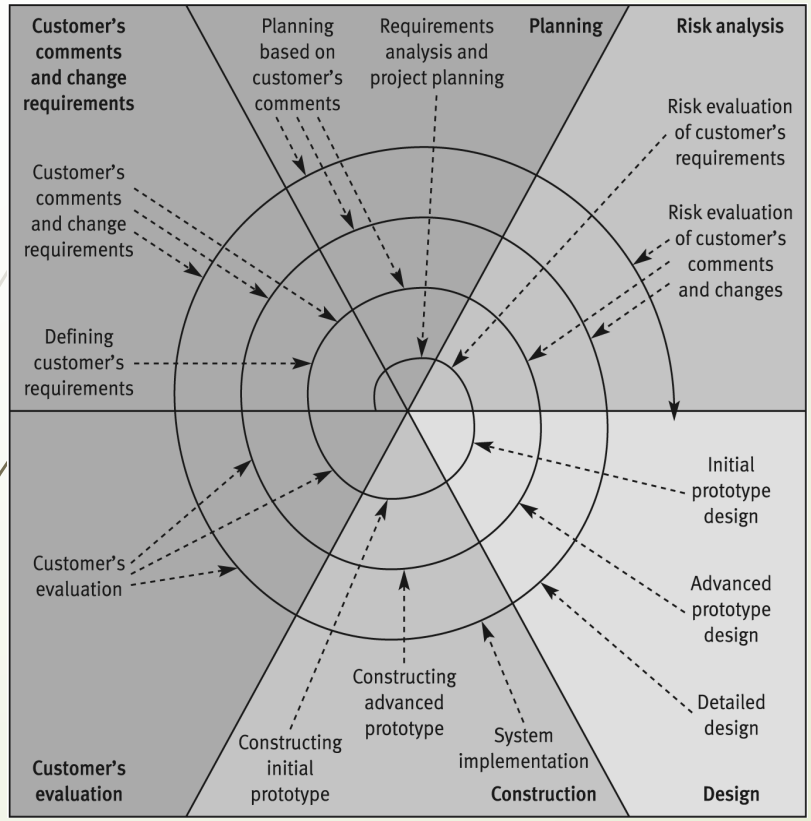
Planning

Risk Analysis / Resolution

Engineering activities (design, code, test…)

Customer Evaluation (errors, changes, new requirements…)

The Spiral Model



Source: After Boehm 1998 (© 1988 IEEE)

Revised Spiral Model provides customer with improved chances for changes;
developer better chances to stay within budget and time.

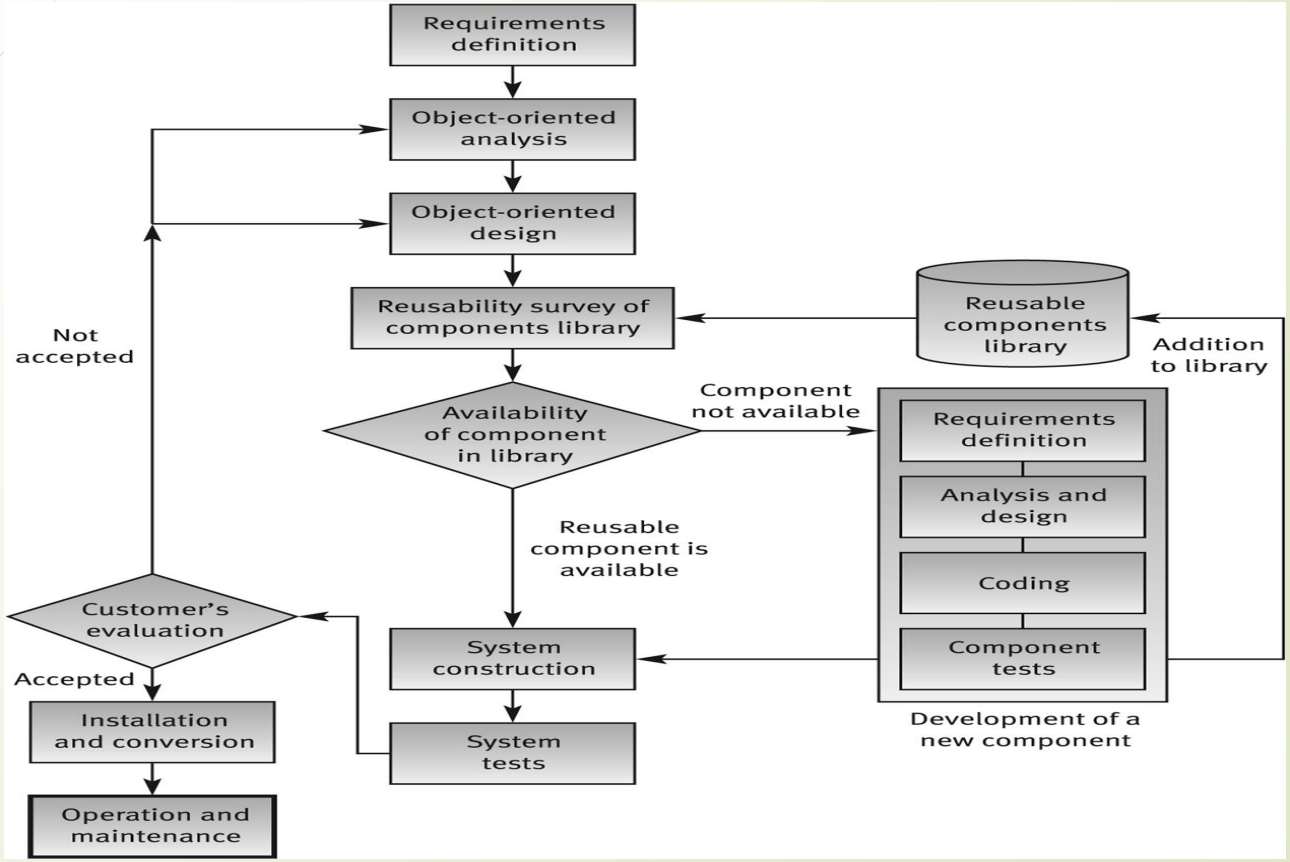
Done by increased emphasis on customer participation and on engineering activities.

Extra sections in spiral dedicated to customer actions and developer engineering

The Object-Oriented Model

- Easy **integration of existing software modules** (objects / components) into newly developed software systems.
- Process begins with OOA and OOD
- Then, acquire **suitable components** from reusable software component libraries (or purchase them).
- Otherwise, develop as needed.
- Can involve adding to repertoire of library components.
- **Economy**: integrating reusable components; much lower cost than developing
- **Improved quality** – using tested components
- **Shorter development times**: integration of reusable software components.

Object Oriented Development Model



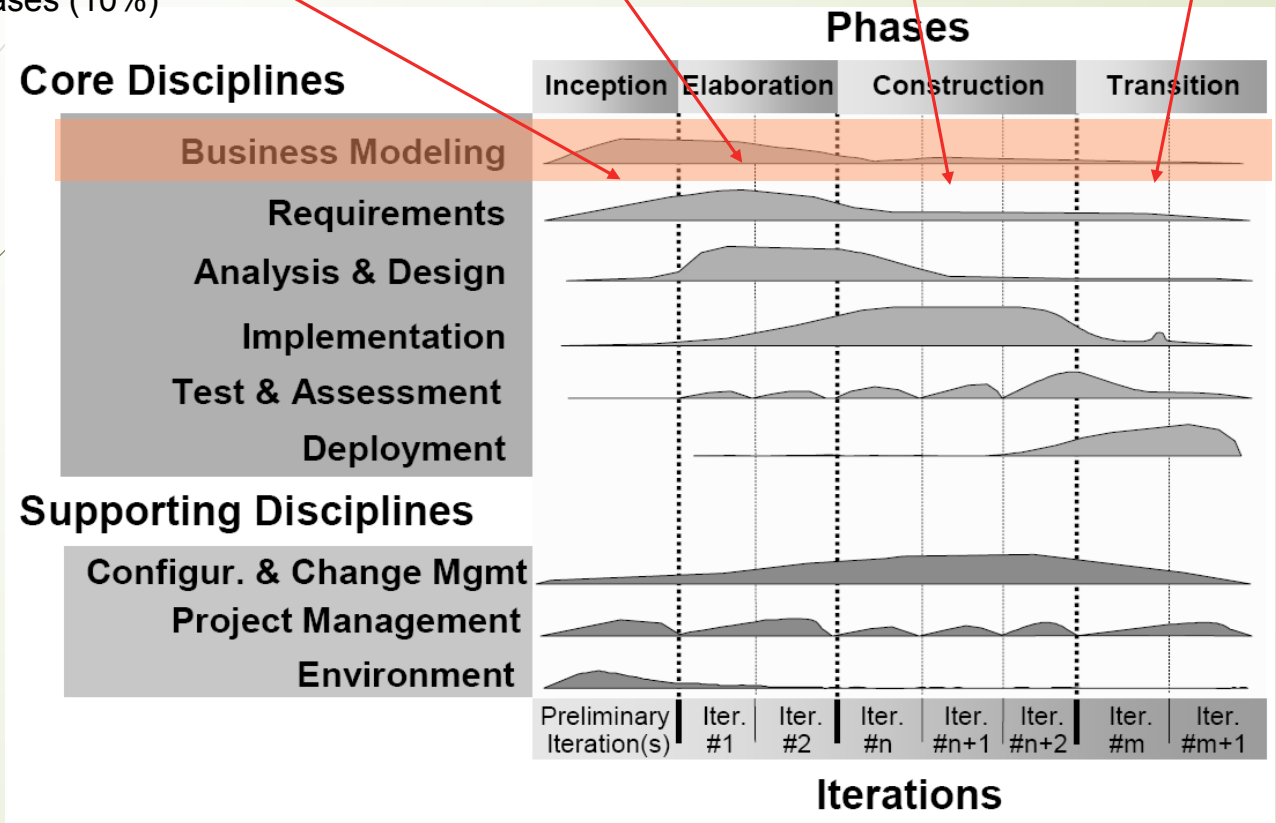
Disciplines, Phases, and Iterations

Identify most of the use cases to define scope, detail critical use cases (10%)

Detail the use cases (80% of the requirements)

Identify and detail remaining use cases

Track and capture requirements changes



Inception Phase

- Overriding goal is obtaining buy-in from all interested parties
  - Initial requirements capture
  - Cost-benefit analysis
  - Initial risk analysis
  - Project scope definition
  - Defining a candidate architecture
  - Development of a disposable prototype
  - Initial use case model (10%-20% complete)
  - First pass at a domain model

Elaboration Phase

- Requirements Analysis and Capture
  - Use Case Analysis
    - Use Cases (80% written and reviewed by end of phase)
    - Use Case Model (80% done)
    - Scenarios
      - Sequence and Collaboration Diagrams
      - Class, Activity, Component, State Diagrams
  - Glossary (so users and developers can speak common vocabulary)
  - Domain Model
    - To understand the problem: the system's requirements as they exist within the context of the problem domain
  - Risk Assessment Plan revised
  - Architecture Document

Construction Phase

- Focus is on implementation of the design
  - Cumulative increase in functionality
  - Greater depth of implementation (stubs fleshed out)
  - Greater stability begins to appear
  - Implement all details, not only those of central architectural value
  - Analysis continues, but design and coding predominate

Transition Phase

- The transition phase consists of the transfer of the system to the user community
  - Includes manufacturing, shipping, installation, training, technical support, and maintenance
  - Development team begins to shrink
  - Control is moved to maintenance team
  - Alpha, Beta, and final releases
  - Software updates
  - Integration with existing systems (legacy, existing versions…)

Agile

SPRINT 1
DEPLOY
TEST
DEVELOP
REVIEW
PLAN
DESIGN
LAUNCH
SPRINT 2
DEPLOY
TEST
DEVELOP
REVIEW
PLAN
DESIGN
LAUNCH
SPRINT 3
DEPLOY
TEST
DEVELOP
REVIEW
PLAN
DESIGN
LAUNCH

Outline

2.1. Software development methodologies

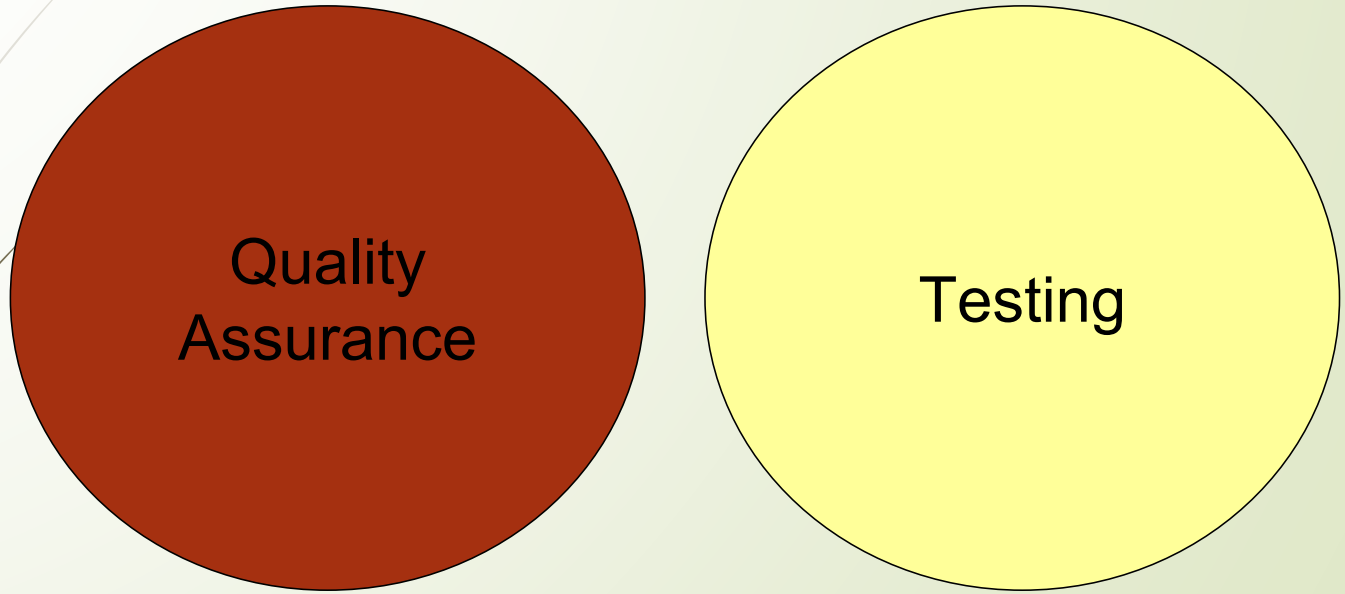
2.2. Testing level

- 2.2.1. Introduction
- 2.2.2. Unit test
- 2.2.3. Integration test
- 2.2.4. System test
- 2.2.5. Acceptance test

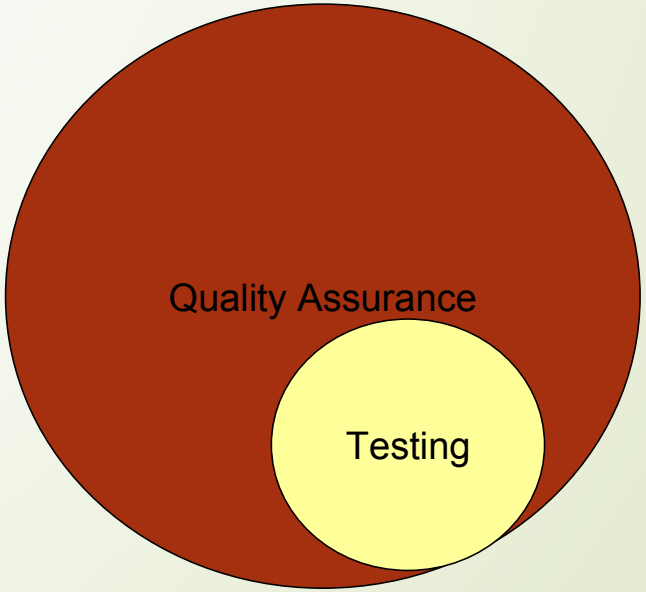
2.3. A model of SQA defect removal effectiveness and cost.

2.4. Create SQA plan

Quality Assurance vs Testing



Quality Assurance vs Testing



Quality Assurance

Multiple activities throughout the dev process

- Development standards
- Version control
- Change/Configuration management
- Release management
- Testing
- Quality measurement
- Defect analysis
- Training

Testing

Also consists of multiple activities

- Unit testing
- Whitebox Testing
- Blackbox Testing
- Data boundary testing
- Code coverage analysis
- Exploratory testing
- Ad-hoc testing
- …

Testing Axioms

- Testing cannot show that bugs do not exist
- Exhaustive testing is impossible for non-trivial applications
- Software Testing is a Risk-Based Exercise. Testing is done differently in different contexts, i.e. safety-critical software is tested differently from an e-commerce site.
- Testing should start as early as possible in the software development life cycle
- The More Bugs you find, the More bugs there are.

Common Error Categories

- Boundary-Related
- Calculation/Algorithmic
- Control flow
- Errors in handling/interpretting data
- User Interface
- Exception handling errors
- Version control errors

Testing Principles

- All tests should be traceable to customer requirements
  - The objective of software testing is to uncover errors.
  - The most severe defects are those that cause the program to fail to meet its requirements.
- Tests should be planned long before testing begins
  - Detailed tests can be defined as soon as the system design is complete
- Tests should be prioritised by risk since it is impossible to exhaustively test a system.
  - Pareto principle holds true in testing as well.

What do we test? When do we test it?

- All artefacts, throughout the development life cycle.
- Requirements
  - Are the complete?
  - Do they conflict?
  - Are they reasonable?
  - Are they testable?

What do we test? When do we test it?

- Design
  - Does this satisfy the specification?
  - Does it conform to the required criteria?
  - Will this facilitate integration with existing systems?
- Implemented Systems
  - Does the system do what is it supposed to do?
- Documentation
  - Is this documentation accurate?
  - Is it up to date?
  - Does it convey the information that it is meant to convey?

The Testing Process

Test Planning

- Test planning involves the establishment of a test plan
- Common test plan elements:
  - Entry criteria
  - Testing activities and schedule
  - Testing tasks assignments
  - Selected test strategy and techniques
  - Required tools, environment, resources
  - Problem tracking and reporting
  - Exit criteria

Test Design and Specification

- Review the test basis (requirements, architecture, design, etc)
- Evaluate the testability of the requirements of a system
- Identifying test conditions and required test data
- Design the test cases
  - Identifier
  - Short description
  - Priority of the test case
  - Preconditions
  - Execution
  - Post conditions
- Design the test environment setup (Software, Hardware, Network Architecture, Database, etc)

Test Execution

- Verify that the environment is properly set up
- Execute test cases
- Record results of tests (PASS | FAIL | NOT EXECUTED)
- Repeat test activities
  - Regression testing

Result Analysis and Reporting

- Reporting problems
  - Short Description
  - Where the problem was found
  - How to reproduce it
  - Severity
  - Priority
- Can this problem lead to new test case ideas?

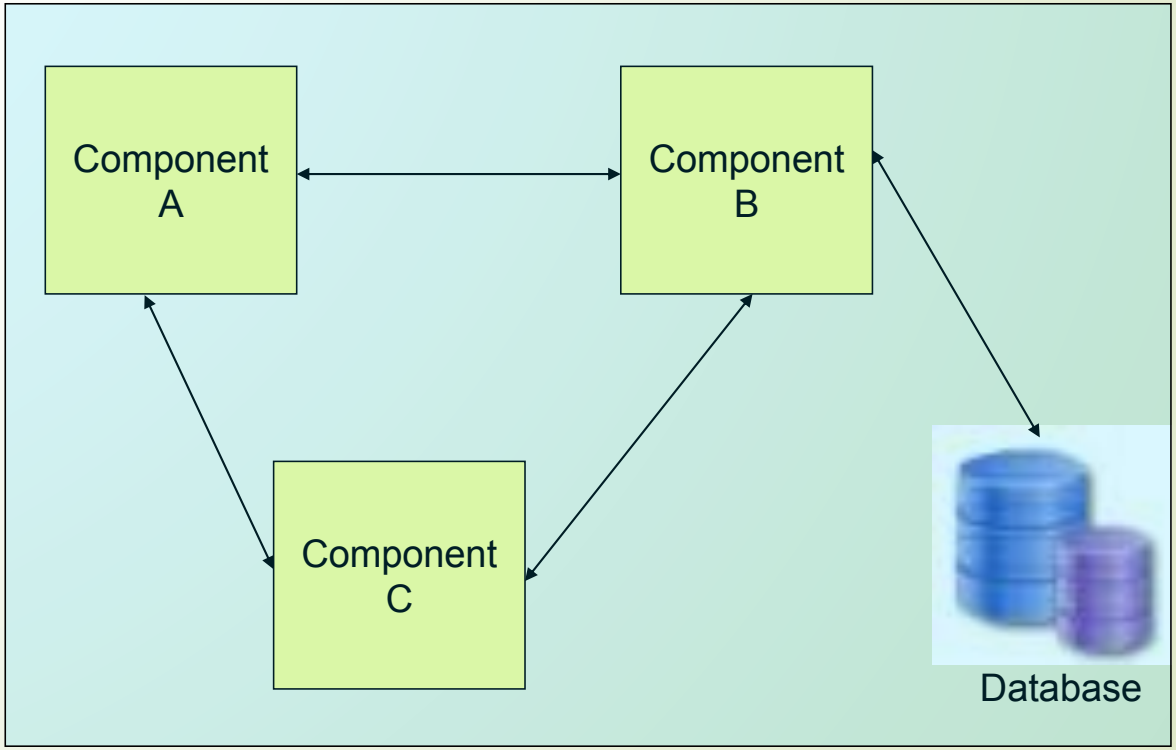
Test Control, Management and Review

- Exit criteria should be used to determine when testing should stop. Criteria may include:
  - Coverage analysis
  - Faults pending
  - Time
  - Cost
- Tasks in this stage include
  - Checking test logs against exit criteria
  - Assessing if more tests are needed
  - Write a test summary report for stakeholders

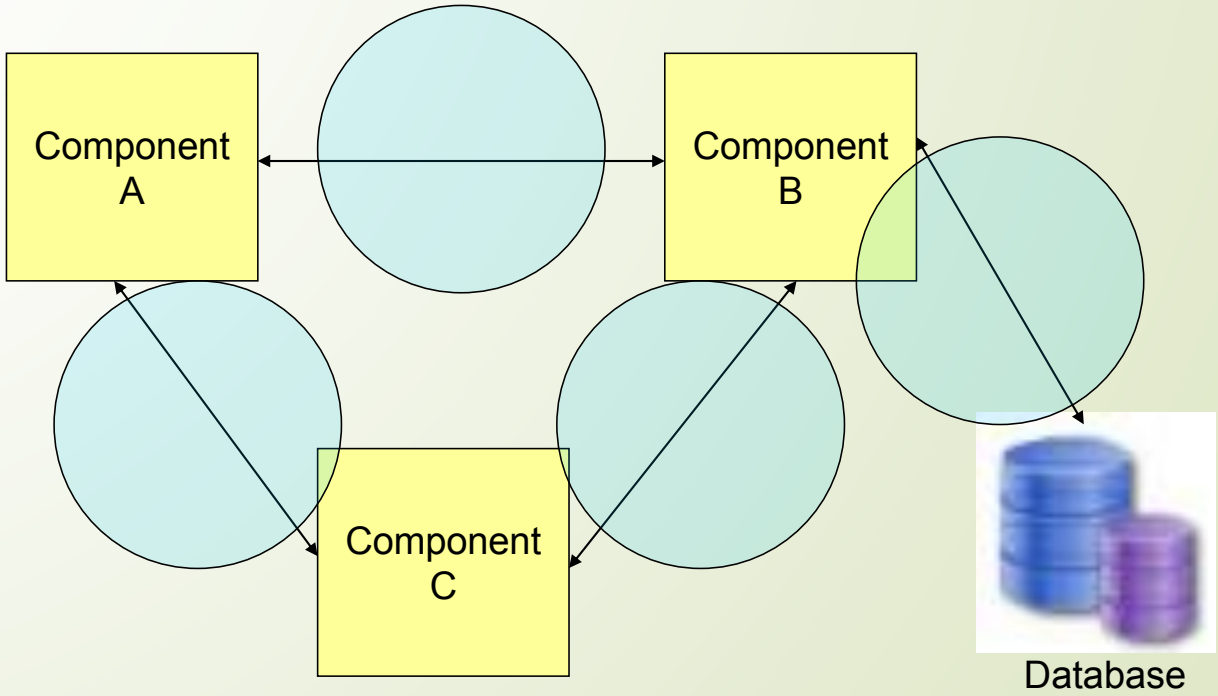
Levels of Testing

- User Acceptance Testing
- System Testing
- Integration Testing
- Unit Testing

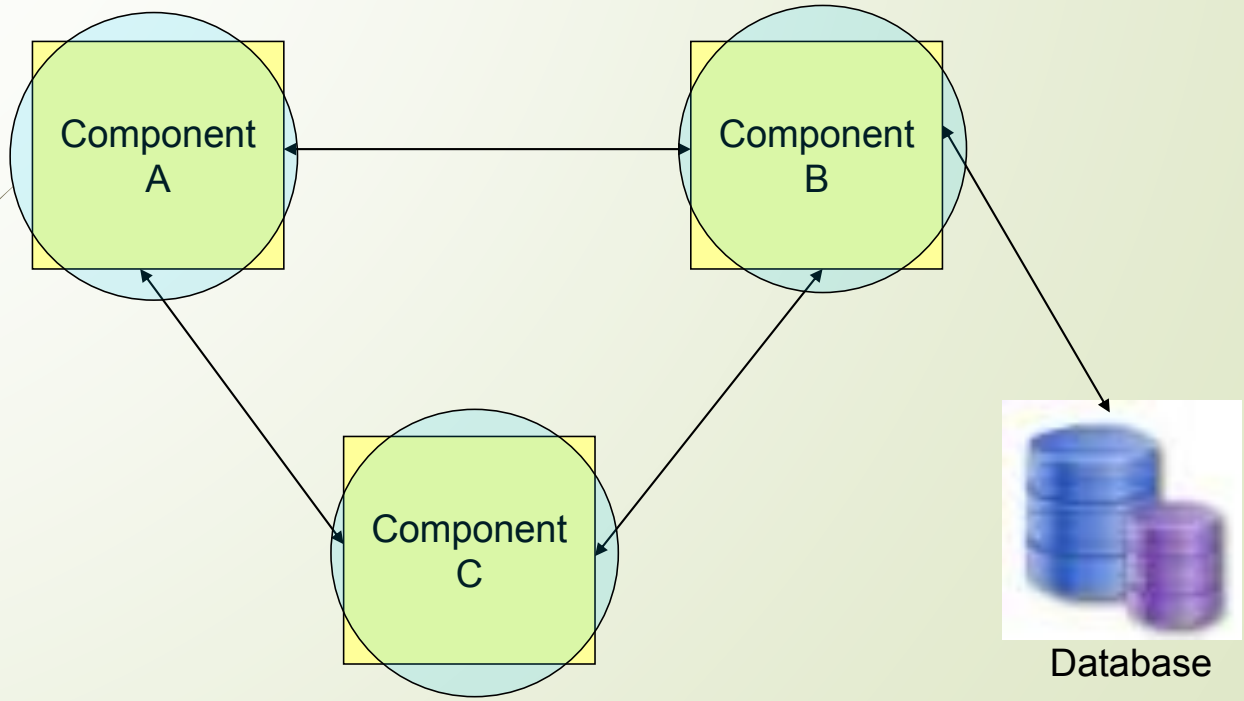
System Testing



Integration Testing



Unit Testing



Outline

2.1. Software development methodologies

2.2. Testing level

- 2.2.1. Introduction
- 2.2.2. Unit test
- 2.2.3. Integration test
- 2.2.4. System test
- 2.2.5. Acceptance test

2.3. A model of SQA defect removal effectiveness and cost.

2.4. Create SQA plan

Model for SQA defect removal effectiveness and cost

The model addresses two quantitative aspects of the SQA planning addressing several defect detection activities.

a.Want to study the SQA plan's total effectiveness in removing project defects, and

a.The total costs of removal of project defects

Note again that SQA activities must be integrated within the project's development plan.

Model for Defect Removal

The data:

Model based on three types of data:

Defect origin distribution
in which phase did the defects occur

Defect removal effectiveness, and
how effective are we at removal of defects?

Cost of defect removal.
how much does it cost per defect per phase

Defect Origin Distribution

- Very consistent over many years:
- Distribution of Defects:
  - Requirements Specs 15%
  - Design 35%
  - Coding / integration 40%
  - Documentation 10%

Defect Removal Effectiveness

- Generally speaking, the percentage of removed defects is lower than the percentage of detected defects, because some corrections are ineffective or inadequate.
- We simply miss some!!
- Others are undetected and uncorrected and passed on to successive development phases.
- Lingering defects coupled with introduced defects in current development phase add up!!!
- For discussion purposes, we will assume the filtering effectiveness of accumulated defects of each quality assurance activity is not less than 40%, that is, each activity removes at least 40% of the incoming defects.

Defect Removal Effectiveness

Removal effectiveness

QA Activity	Average defect filtering
effectiveness rate	
requirements specs review	50%
design inspection	60%
design review	50%
code inspections	65%
unit test	50%
Unit test > code review	30%
integration test	50%
system tests / acceptance	50%
documentation review	50%

Cost Removal

- Removal of defects differs very significantly by development phase.
- Cost are MUCH greater in later development phases.

- Note: In general, defect removal data is not commonly available.
- Most agree with the data based on key studies. (next slide)

The Model

Model is based on following assumptions:

- Development process is linear, sequential following waterfall model
- There is a number of new defects introduced each phase
- Review and test software quality assurance activities serve as filters, removing a percentage of defects and letting the rest pass to next development phase as we saw three slides back.
- At each phase, incoming defects are the sum of those not removed plus new defects in current phase
- Cost of defect removal is calculated for each SQA activity by multiplying the number of defects removed by the relative cost of removing a defect. (see table, previous slide)
- Remaining defects are passed to the customer. (this is the heaviest cost for defect removal)

Costs of Defect Removal

- But we can do better using a comprehensive quality assurance plan with more activities, and hence better filtering.

- The comprehensive quality assurance plan (comprehensive defects filtering system) accomplishes the following:

- 1. Adds two quality assurance activities so that the two are performed in the design phase as well as in the coding phase
  - We have a Design Inspection and a Design Review vice Design, and
  - Code Inspections and unit test vice simple Unit Test
- 2. Improves the 'filtering' effectiveness of other quality assurance activities.

Outline

2.1. Software development methodologies

2.2. Testing level

- 2.2.1. Introduction
- 2.2.2. Unit test
- 2.2.3. Integration test
- 2.2.4. System test
- 2.2.5. Acceptance test

2.3. A model of SQA defect removal effectiveness and cost.

2.4. Create SQA plan

SQA Plan

- The software quality assurance plan is one of the most important plans that should be prepared before embarking on a software development project.
- The following details are recorded in the software quality assurance plan:

1. **Standards**—Include coding guidelines, design guidelines, testing guidelines, etc. selected for use in the project. These standards ensure a minimum level of quality in software development as well as uniformity of output from the project resources.

2. **Quality control activities**—Proposed activities for the project include code walkthrough, requirements and design review, and tests (unit testing, integration testing, functional testing, negative testing, endto-end testing, system testing, acceptance testing, etc.).

SQA Plan

4. **Procedures and events that trigger causal analysis**—Include failures, defects, and successes.

5. **Audits**—To analyze the exceptions in the project so that necessary corrective and preventive actions are taken to ensure the exceptions do not recur in the project.

6. **Institute of Electrical and Electronics Engineers Standard 730**—Gives details on how to prepare a quality assurance plan, including a suggested template.

SQA Plan

Details of the following standards should be included in the software quality assurance plan to guide project personnel in carrying out their assignments effectively and with the desired levels of productivity and quality:

- Coding standards for the programming languages used in the project
- Database design standards
- Graphical user interface design standards
- Test case design standards
- Testing standards
- Review standards
- Organizational process reference

SQA Plan

The following specifications of quality levels (quality metrics) for the project should be stated in the software quality assurance plan:

- Defect injection rate
- Defect density
- Defect removal efficiency for various quality assurance activities
- Productivity for various artifacts of the project
- Schedule variances

SQA Plan

The following quality control activities proposed to be implemented in the project should be included in the software quality assurance plan:

- Code walkthrough
- Peer review
- Formal review
- Various types of software tests that would be carried out during project
- execution, which at a minimum should include the following:
  - Unit testing
  - Integration testing
  - System testing
  - Acceptance testing

SQA Plan

It also should contain the schedules for the following audits proposed for the project:

- Periodic conformance audits
- Phase-end audits
- Investigative audits (and criteria)
- Delivery audits